

ltm_ft

TabFM v1.0.0 · Apple M4 · NVIDIA L4 on GKE

Fine-tuning a tabular foundation model laptop vs GPU

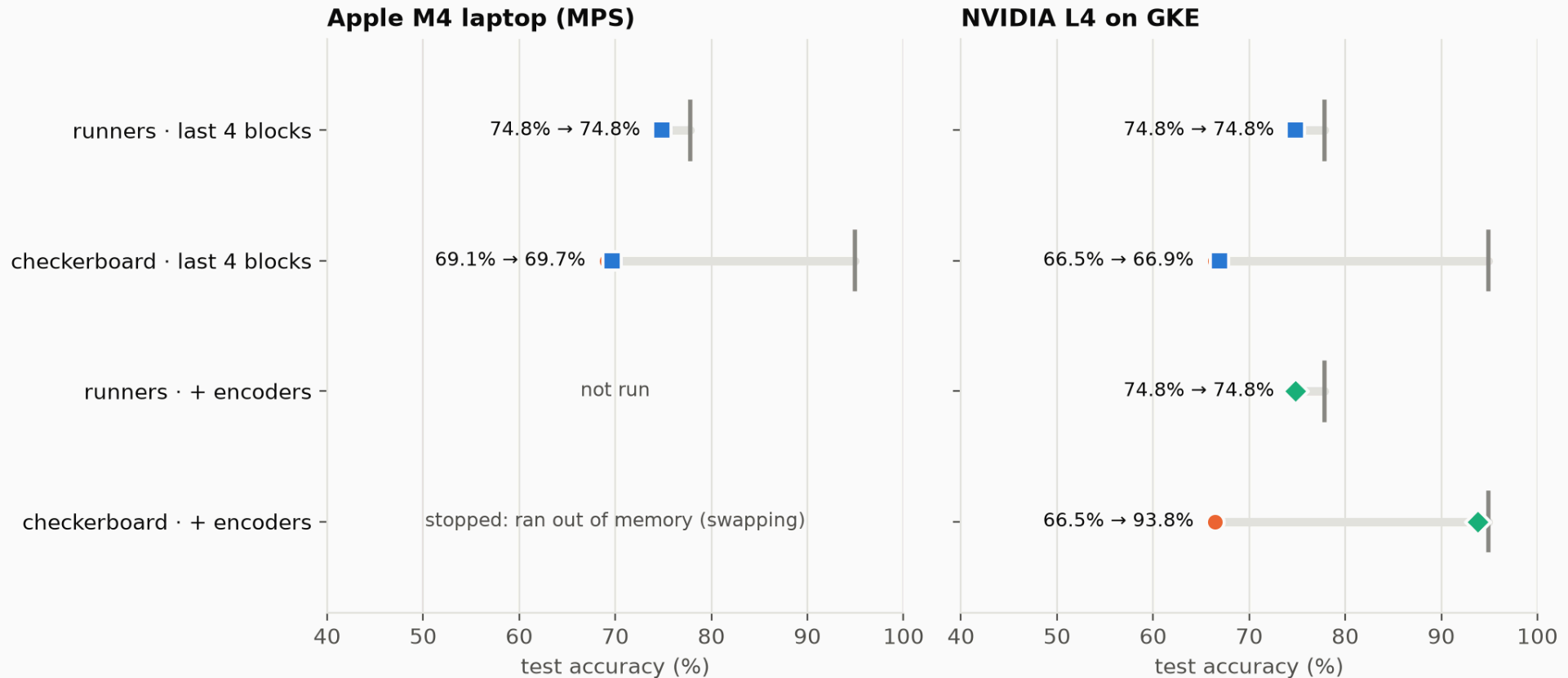
Reading not your game? Slides instead. Zero-shot vs fine-tuned TabFM on synthetic tables where the best achievable score is known, on a laptop and as a Kubernetes Job.

Gleb Lukicov · github.com/glukicov/ltm_ft

Results at a glance

Only training the encoders moved TabFM far

Zero-shot vs fine-tuned TabFM, same 3,000 test rows per task



Same 3,000 test rows per task. Encoders on the L4: checkerboard 66.5% → 93.8% (ceiling 94.9%); other rows moved under a point.

The question

TabFM already learns in context, so what do training and a GPU add?

TabFM predicts a row from labelled context rows in one forward pass, with no training step. Fine-tuning keeps that interface and trains it on your table. We ran the same code on an Apple M4 laptop and on an NVIDIA L4 in GKE.

Before: glukicov/ltn and ltn_serve

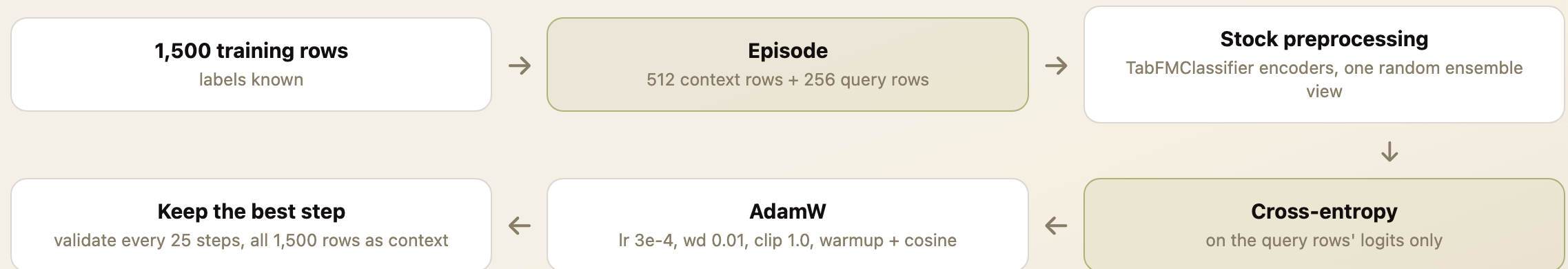
Zero-shot TabFM matched a trained CatBoost model with no training; ltn_serve then served it on GKE with an L4 pool that scales from zero.

The paper: arXiv 2506.08982

Full fine-tuning makes TabPFNV2 state of the art on academic benchmarks; on small datasets the gains are often statistically insignificant.

The recipe

Every step is one in-context episode, scored on its query rows



src/ltn_ft/finetune.py · a test asserts episode logits equal the stock classifier's

The recipe

The training step calls the unmodified model

src/lstm_ft/finetune.py

```
loss = episode_loss(model, episode)
loss.backward() # type: ignore[no-untyped-call]
master.gradients_to_masters()
torch.nn.utils.clip_grad_norm_(master.masters, config.grad_clip)
optimizer.step()
scheduler.step()
optimizer.zero_grad(set_to_none=True)
master.masters_to_params()
```

No custom forward: frozen parameters do not require gradients, so autograd records nothing until the first trainable module.

Master weights: gradients move to float32 copies, AdamW updates those, and the rounded values go back into the bf16 model.

What to train

1.64B parameters do not fit AdamW in 24 GB, so we train a slice

What is trained	Trainable	Apple M4, 24 GB unified	NVIDIA L4, 24 GB
Everything	1.64B	~26 GB for weights, gradients and two Adam moments: does not fit either	
Last 4 of 24 ICL blocks + head	277M	3.87 s/step, 11.0 GB	0.45 s/step, 8.8 GB
... + row/column encoders	297M	36.7 s/step, 17.9 GB; swaps with other apps	0.74 s/step, 14.9 GB

1.62B of the 1.64B parameters sit in the in-context-learning transformer. Training the encoders needs a backward pass through all 24 blocks. M4 memory is what the MPS driver held; L4 is CUDA peak allocation.

The trap

Same checkpoint, same rows: 59.8%, 65.0% or 70.2%

Checkerboard, zero-shot, 500 validation rows	Log loss	ROC AUC	Accuracy
float32 (dtype=None), M4 and L4 identical	0.6495	0.6658	59.8%
bfloat16 on the L4 (CUDA)	0.6323	0.7244	65.0%
bfloat16 on the M4 (MPS)	0.6077	0.7749	70.2%

Why it matters: "upcast the weights you train to float32" is the textbook move, and here it changes the model before the first gradient step. bfloat16 also differs by device, so zero-shot and fine-tuned must be compared on the same machine.

The fix

Keep the model in bf16 and the optimiser's copies in float32

src/ltn_ft/finetune.py · class Float32Master

```
def __init__(self, params: list[nn.Parameter]) -> None:
    self.params = params
    self.masters = [p.detach().float().clone() for p in params]

def gradients_to_masters(self) -> None:
    for p, m in zip(self.params, self.masters, strict=True):
        m.grad = None if p.grad is None else p.grad.float()
        p.grad = None

@torch.no_grad()
def masters_to_params(self) -> None:
    for p, m in zip(self.params, self.masters, strict=True):
        p.copy_(m)
```

Step 0 is the stock model: a test asserts the prepared model's predictions equal the untouched checkpoint's.

Nothing to convert: the fine-tuned result is a plain bf16 TabFM the stock `TabFMClassifier` uses directly.

Realistic table

On the runners table, nothing we trained beat zero-shot

3,000 test rows (ceiling 77.8%)	Zero-shot	Fine-tuned	Step kept
M4, last 4 blocks	74.8%	74.8%	0
L4, last 4 blocks	74.8%	74.8%	0
L4, + encoders	74.8%	74.8%	0

Every validation check after a gradient step was worse than zero-shot, so early stopping kept the original weights each time. The messy 21-column table is the one from glukicov/ltm.

Realistic table

TabFM ignored 100 appended columns of pure noise

0.4651 → 0.4672

validation log loss

76.4% → 75.8%

validation accuracy

0.8554 → 0.8556

validation ROC AUC

Zero-shot on the M4, 21 real columns vs the same rows with 100 extra noise columns (`--noise-columns 100`). Distractors were not a weakness to fine-tune away either.

A table TabFM is bad at

The checkerboard leaves zero-shot TabFM far below the ceiling

Ten uniform features. The label depends only on which square of a 6×6 board the first two fall in: $P(\text{yes})$ is $\text{sigmoid}(+3)$ on white squares, $\text{sigmoid}(-3)$ on black.

Why it is hard

Neither feature is informative alone, and a row's nearest neighbours in ten dimensions are mostly on other squares.

The gap, data seed 7

Test accuracy: zero-shot 69.1% on the M4, 66.5% on the L4; ceiling 94.9%.

Same recipe, two machines

The last 4 blocks give the same small, significant gain on both

Checkerboard, seed 7, 3,000 test rows	Zero-shot	Fine-tuned	Δ log loss [95% CI]	Δ accuracy [95% CI]
Apple M4 (MPS)	69.1%	69.7%	-0.021 [-0.024, -0.018] significant	+0.6 [-0.6, +1.8]
NVIDIA L4 (CUDA)	66.5%	66.9%	-0.017 [-0.020, -0.014] significant	+0.5 [-0.9, +1.8]

Better probabilities, the same decisions. The zero-shot starting points differ by device because bfloat16 kernels differ (slide 7).

Moving the job

One image, an L4 pool that scales from zero, one Job per batch

up.sh

Cloud Build image from uv.lock +
CUDA torch; cluster



finetune.sh

lrm-ft commands become one Job
on the L4



collect.sh

copy outputs/, write the timeline,
delete the Job



down.sh

delete cluster, images, uploads

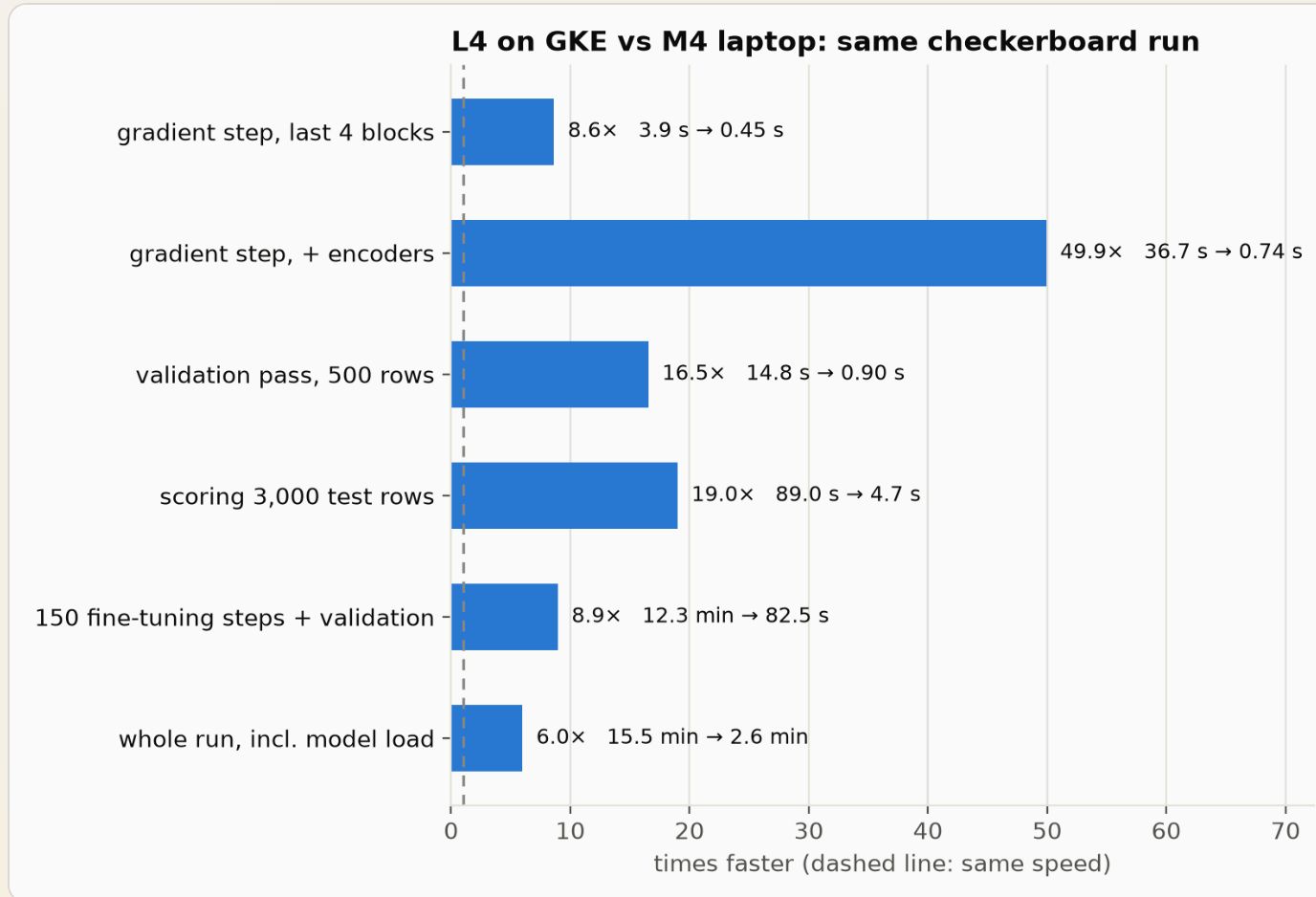
k8s/gke/job-finetune.yaml

```
tolerations:  
  - key: nvidia.com/gpu  
    operator: Exists  
    effect: NoSchedule  
nodeSelector:  
  cloud.google.com/gke-accelerator: nvidia-l4
```

No weights in the image: the pod downloads TabFM from Hugging Face inside GCP; the weights are non-commercial.

Quota decides the GPU: the project allows one GPU and no A100s, so 1x L4 (24 GB) on a g2-standard-8.

The L4 fine-tunes 9× faster, and ~50× for the encoders



Same checkerboard run: 150 steps + validation 12.3 min → 82.5 s; whole run 15.5 min → 2.6 min. On the laptop the encoder variant swapped and was stopped.

Laptop vs GPU

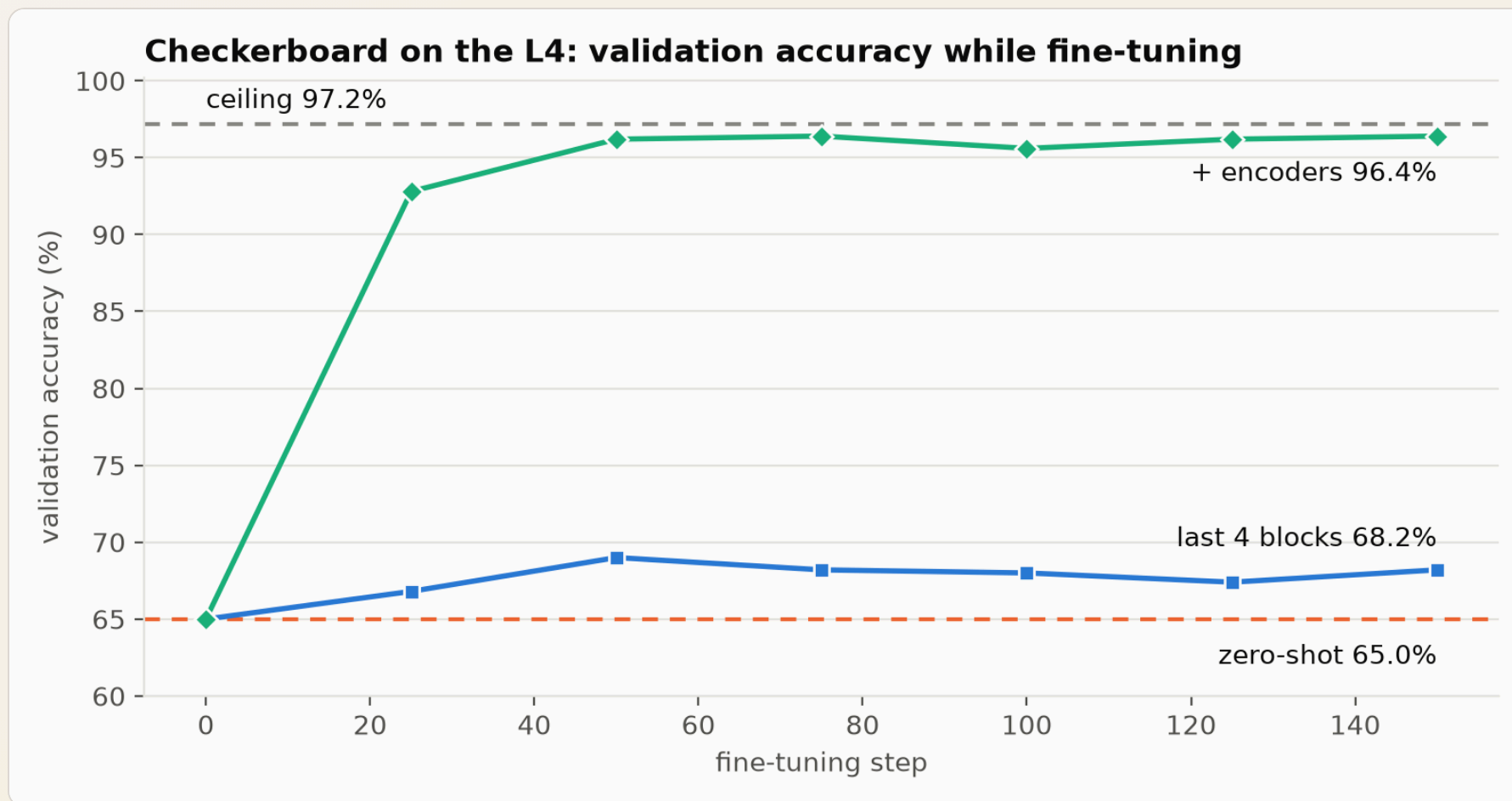
Kubernetes costs minutes once, then seconds; a run costs ~4 cents

First Job, from creation	Seconds	List price, us-central1	USD
GPU node exists	53	g2-standard-8 (1x L4), per hour	0.854
Pod scheduled	104	... Spot, per hour	0.512
Image pulled (2 min 3 s)	228	One checkerboard run (171 s)	~0.04
Container started	228	Session: L4 node 1.28 h + system node	1.09 + 0.18
Later Jobs on the warm node	~1	20 runs, 1.05 GPU-hours busy	0.89

One-off setup: image build 8 min 4 s, cluster 4 min 5 s. Excludes the GKE management fee and Cloud Build minutes.

The variant the laptop could not run

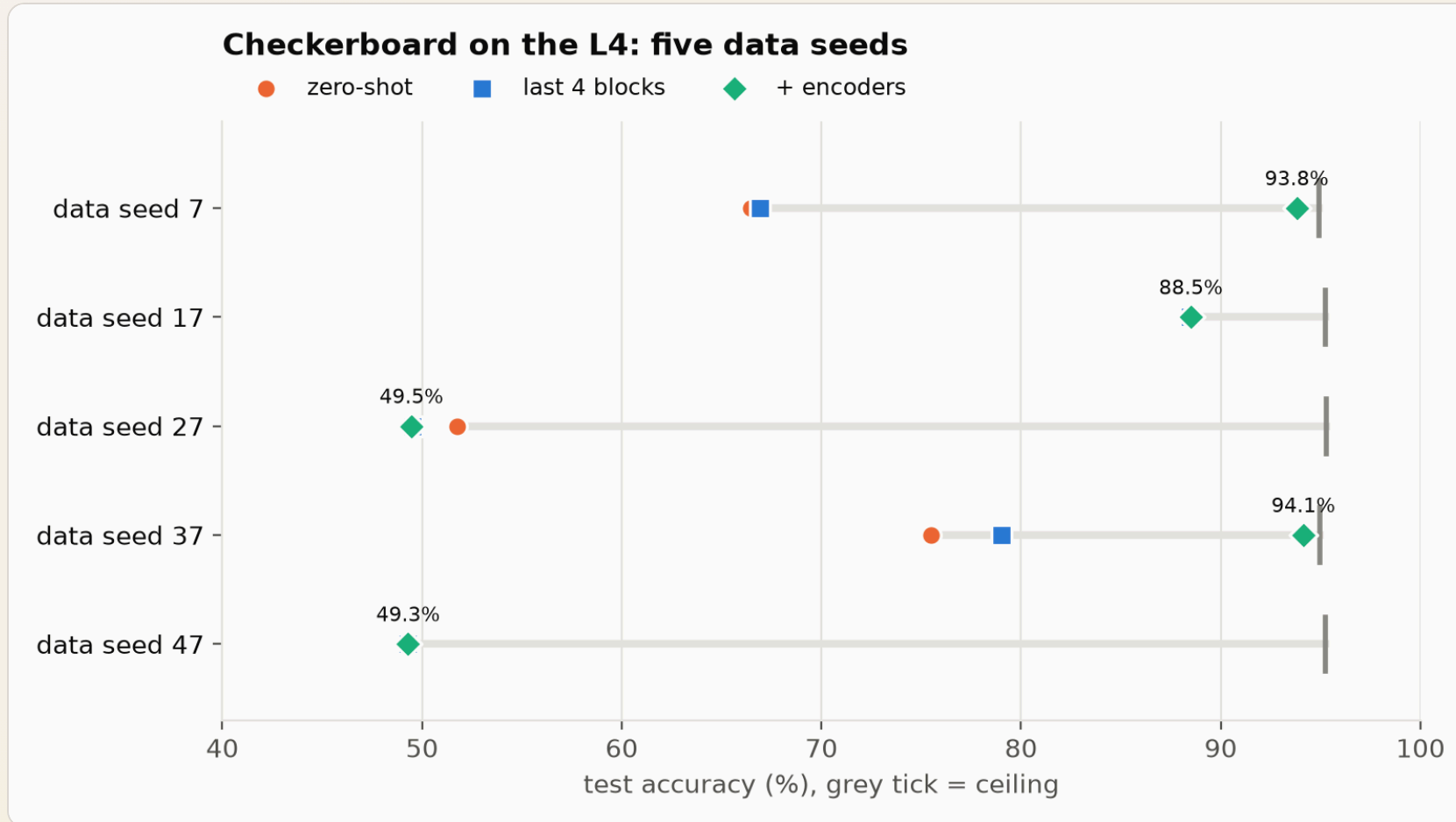
Training the encoders took the checkerboard from 66.5% to 93.8%



Test, data seed 7: zero-shot 66.5%, last 4 blocks 66.9%, + encoders 93.8% (log loss 0.623 $\rightarrow$ 0.224; ceiling 94.9%, 0.202).

Five seeds later

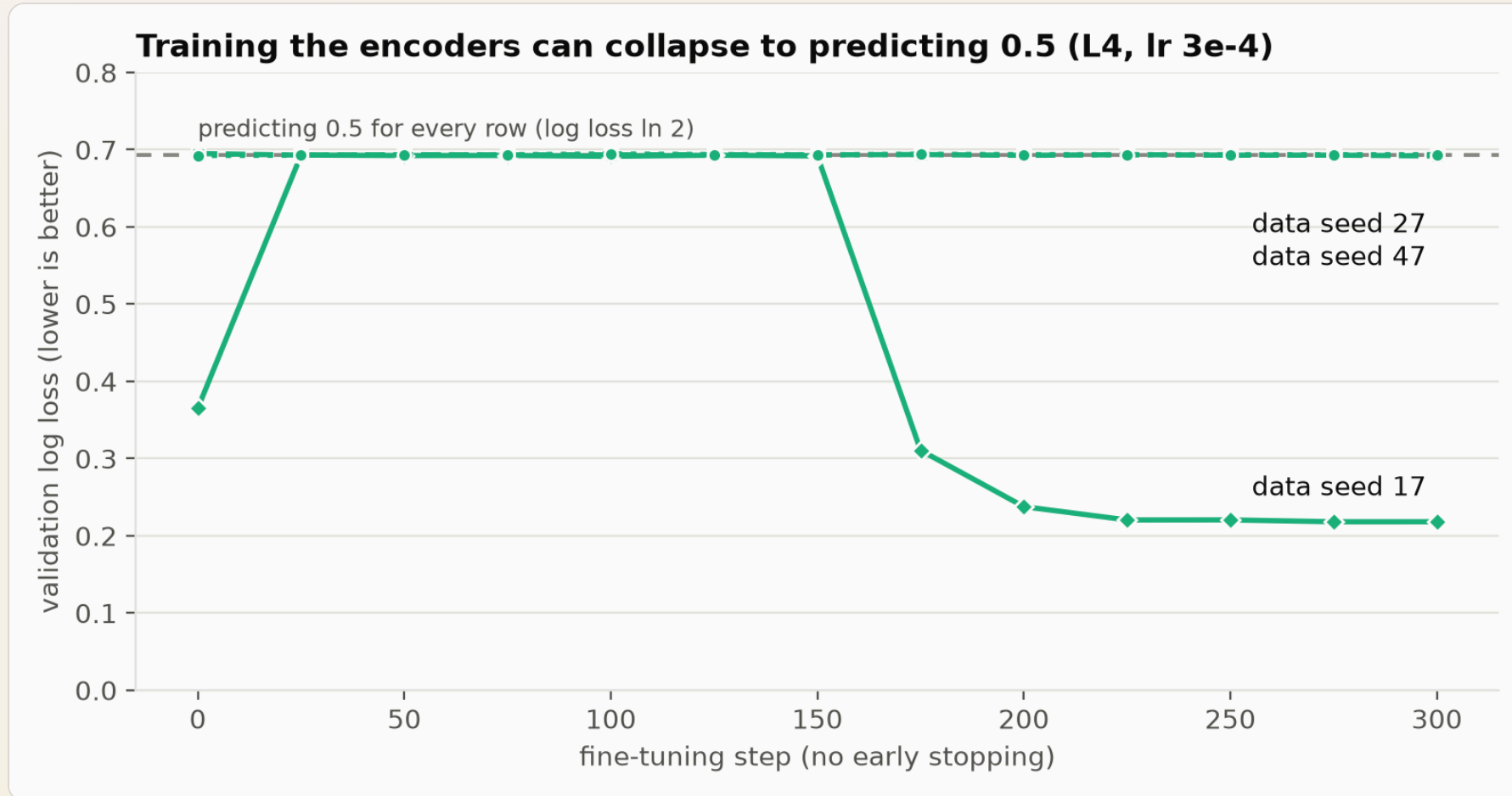
The encoder variant reaches the ceiling on 2 of 5 seeds



Seeds 7 and 37 reach 93.8% and 94.1%. Seeds 17, 27 and 47 stay at zero-shot; zero-shot itself ranges from 49.3% to 88.5%. 20 minutes on the L4.

Five seeds later

When it fails, training collapses to predicting 0.5



Seed 17 collapses for 150 steps, then recovers to 94.2% validation accuracy by step 300. Seeds 27 and 47 never leave ln 2 at lr 1e-4, 3e-4 or 1e-3.

Honest limits

Read these effects as a floor, and the big one as fragile

Scale

1,500 training rows, at most 150 steps (300 for diagnostics), 17–18% of the model. The paper trains the whole model on an 80 GB GPU on larger datasets.

Seeds

Five seeds on one synthetic task, chosen because zero-shot TabFM is weak on it; one seed for the others.

Hardware

One L4 is the largest GPU the project's quota allows. Laptop timings vary with other load: the same test pass took 52 s and 126 s.

Licence

TabFM weights are non-commercial: none are committed or baked into images. Code is MIT.

non-commercial weights

Closing

Takeaways and try it

What we found

- ✓ A dedicated GPU pays off the moment you iterate: speed made the seed check affordable
- ✓ A laptop runs out of memory before it runs out of speed
- ✓ Run more than one seed before you believe a fine-tuning win
- ✓ Precision and device are part of the model; check the headroom first

Start here

- `README.md` : results, cost and caveats
- `outputs/m4` , `outputs/l4` : every number on these slides
- `k8s/gke` : the cluster and Job scripts

```
uv run ltm-ft run --task checkerboard --train-encoders --name m4/enc
k8s/gke/up.sh
k8s/gke/finetune.sh "run --task checkerboard --train-encoders --name l4/enc"
k8s/gke/down.sh
```

github.com/glukicov/ltm_ft · Questions?